
The Twelve Exercises

Put the security back into a deliberately insecure cloud-robotics stack — one control at a time, each one proved before it is fixed and proved again afterwards.

12 exercises

Prerequisites & skills

Setup & run, no Docker

Architecture overview

One infographic per exercise

Companion to `secure_robotstore/README.md`. Every exercise has a working reference implementation in the main repository, named on its own page — read it *after* your attempt, not before.

Nothing in this sandbox is secure. That is the point.

How to work through this

Read this page once. It is the difference between applying security controls you half-understand and applying the right ones, in the right place, for the right reason.

THE METHOD – THREE STEPS, NEVER TWO

Prove it. Fix it. Prove it again. Every exercise opens with a command that demonstrates the weakness against the running stack, and closes with that same command failing. A fix you cannot demonstrate is a fix nobody can trust — and several of the weaknesses here are completely invisible from the user interface. If you cannot show the before, you have not earned the after.

Three rules that apply to every exercise

1 • Keep the application working. A security change that silently stops the data flowing is not a pass. The most common way to fail these is to lock out the *legitimate* clients along with the attacker — remember that the agent and the backend are MQTT clients too. After every change:

```
# restart whichever terminal you changed – broker, backend or agent – then:  
curl -s localhost:8001/health/detail | python3 -m json.tool  
# then open http://localhost:3001 – both windows should still tick at 1 Hz
```

2 • Read the reference implementation afterwards, not before. Every exercise names a file in the main repository that solves the same problem properly. The point is to arrive at the reasoning yourself; the comparison is the check on it, and it is much more useful once you have something to compare.

3 • Match the reasoning, not the line count. The main stack solves these at fleet scale — many robots, per-device identities, token revocation, hash-chained audit. This sandbox has one device and two data types. Where the reference is more elaborate than this needs, its own docstring usually says why.

BEFORE YOUR FIRST COMMAND

Everything here runs against the sandbox on your own machine. Nothing in this book is ever run against a production deployment, against a robot that can move while a person is within reach, or against any system you have not been given written authorisation to test. If you find something that reaches outside the sandbox, that is the moment to stop and tell someone — not the moment to confirm it.

The suggested order — and why it is this order

#	EXERCISE	WHY IT COMES HERE
1	MQTT authentication	Nothing else can be scoped until the broker can tell clients apart.
2	MQTT topic ACLs	Identity without authorisation still lets any client write any topic.
3	Credential checking	The operator half of identity; small code, large consequence.
4	Real tokens	A credential check is only useful if the resulting session is unforgeable.
5	Protecting endpoints	The one that matters most. 3 and 4 are worth nothing without it.
6	Login rate limiting	Now that the login is correct, make guessing at it expensive.
7	Input size limits	Availability: one client must not be able to stall everyone.
8	CORS & headers	The browser-facing edge, cheap to fix and hard to justify skipping.
9	TLS	Protects the credentials added in 1 — without it, 1 is undone on every connect.
10	Container hardening	Bounds what any successful intrusion is able to become.
11	Secrets	Stops a known-default credential ever reaching a real deployment.
12	Audit logging	When prevention eventually fails, this is what lets you reconstruct it.

Prerequisites & skills

What you need to know before starting, why this particular system needs it, and where each skill shows up in the exercises. Nobody is expected to arrive strong in all twelve.

Essential means an exercise will be confusing without it. **Helpful** means you can learn it as you go, on the exercise that needs it. If you are missing something marked essential, say so at the start — a day spent closing the gap is cheaper than three weeks of working around it. The line under each card is a quick way to check yourself.

Linux command line

ESSENTIAL

WHAT IT IS

Navigating a shell: files and paths, processes, pipes and redirection, reading logs, environment variables.

WHY THIS SYSTEM NEEDS IT

There is no graphical interface anywhere in this system. Every service is started, inspected and debugged from a terminal, and the robot itself is reached over SSH.

WHERE YOU USE IT HERE

Constantly. `curl`, `grep`, `ss`, `psql`, `mosquitto_sub` — and every “Verify” step in this book is a shell command.

```
► grep -rn "INTERN TASK" secure_robotstore | wc -l
```

Package managers & virtual environments

ESSENTIAL

WHAT IT IS

Installing software with a package manager (`apt`, `brew`, `npm`), and keeping a project's Python packages in a *virtual environment* of its own.

WHY THIS SYSTEM NEEDS IT

You will install and run five separate programs by hand — a broker, two databases, a Python backend and a Node frontend. Nothing is hidden behind an image, so what breaks is always something you can see and fix.

WHERE YOU USE IT HERE

Setup, and then daily. The system runs in four terminals; two of them must be inside the project's virtual environment or nothing imports.

```
► source .venv/bin/activate · pip list · npm ls --depth=0
```

TCP/IP, ports & sockets

ESSENTIAL

WHAT IT IS

How a process listens on a port, the difference between binding `127.0.0.1` and `0.0.0.0`, and how a packet finds its way between two machines.

WHY THIS SYSTEM NEEDS IT

A large share of real vulnerabilities reduce to “something was listening where nobody expected it”. You cannot spot that without knowing what a listening socket is.

WHERE YOU USE IT HERE

Exercise 10 rebinds the databases and broker to loopback. Understanding why that matters — and why it is *not* enough on a robot sitting on campus Wi-Fi — is the point.

```
► ss -tlnp | grep -E '1884|5433|6380'
```

MQTT publish / subscribe

ESSENTIAL

WHAT IT IS

A lightweight messaging protocol for devices: clients publish to named *topics*, a *broker* delivers to everyone subscribed. Nobody addresses anybody directly.

WHY THIS SYSTEM NEEDS IT

MQTT is the only road between the cloud and the robot. Every command, every reading, every heartbeat crosses it — so it is also the single best place to enforce who may say what.

WHERE YOU USE IT HERE

Exercises 1, 2, 7 and 9 are all broker work. You will live inside `mosquitto_pub` and `mosquitto_sub`.

```
► mosquitto_sub -h localhost -p 1884 -t 'devices/#' -v
```

HTTP & REST

ESSENTIAL

WHAT IT IS

Request methods, status codes, headers and bodies — and the idea that the server decides, not the client.

WHY THIS SYSTEM NEEDS IT

The operator API is HTTP. Status codes are how every access-control decision announces itself: 200 versus 401 versus 429 is the entire result of several exercises.

WHERE YOU USE IT HERE

Exercises 3, 5, 6 and 8. Exercise 8 is nothing but response headers.

```
► curl -s -o /dev/null -w '%{http_code}\n'
localhost:8001/api/int/latest
```

WebSockets

HELPFUL

WHAT IT IS

A long-lived, two-way connection that starts as an HTTP request and is then upgraded, so the server can push data without being asked.

WHY THIS SYSTEM NEEDS IT

Telemetry reaches the browser live rather than by polling. Sockets also break the usual authentication pattern, which is what makes exercise 5 interesting.

WHERE YOU USE IT HERE

Exercise 5. The crux: a browser cannot set custom headers on a WebSocket handshake, so `Authorization: Bearer` is simply unavailable to you.

```
► python3 -c "import websockets" · then connect
to /ws/int_api
```

Python 3 & asyncio

ESSENTIAL

WHAT IT IS

Reading and writing modern Python; `async` / `await`, the event loop, and the difference between a coroutine and a thread.

WHY THIS SYSTEM NEEDS IT

Both the backend and the agent are asynchronous Python. More importantly, the single event loop is *why* one oversized message can stall every connected browser at once.

WHERE YOU USE IT HERE

Every code change you make. Exercise 7 cannot be understood at all without knowing what blocking the loop means.

```
► Read store.py and say what runs on which
thread
```

SQL & PostgreSQL

HELPFUL

WHAT IT IS

Tables, queries, and parameterised statements — placeholders rather than string-concatenated values.

WHY THIS SYSTEM NEEDS IT

Postgres is the system of record. It is also where you will prove that a forged message really did become a permanent row.

WHERE YOU USE IT HERE

Verification throughout. Exercise 3 adds a users table; exercise 12 adds the audit table. The existing inserts are parameterised on purpose — keep them that way.

```
► psql -h localhost -p 5433 -U robotstore -d
robotstore -c '\dt'
```

Redis & key-value thinking

HELPFUL

WHAT IT IS

An in-memory store with expiring keys and atomic operations such as `INCR`, `SETNX` and `GETDEL`.

WHY THIS SYSTEM NEEDS IT

Anything that is live, countable or short-lived lives here: the latest reading, a login-attempt counter, a single-use ticket. Atomicity is what makes those correct under concurrency.

WHERE YOU USE IT HERE

Exercise 6's rate limiter is `INCR` plus `EXPIRE`. Exercise 5's ticket is `GETDEL` — read and delete in one operation, so it cannot be used twice.

```
► redis-cli -p 6380 KEYS '*'
```

Cryptography fundamentals

ESSENTIAL

WHAT IT IS

The difference between *hashing* (one-way), *encryption* (reversible with a key) and *signing* (proves origin) — plus salts and constant-time comparison.

WHY THIS SYSTEM NEEDS IT

Choosing the wrong one of the three is the most common serious mistake in this whole subject. “We encrypt passwords” is a sentence that describes a bug.

WHERE YOU USE IT HERE

Exercise 3 hashes. Exercise 4 signs. Exercise 9 encrypts. Being able to say which you need, and why, is more than half of each.

```
► Explain why bcrypt is deliberately slow and
SHA-256 is not
```

TLS & certificates

HELPFUL

WHAT IT IS

How two parties agree on an encrypted channel, what a certificate authority vouches for, and how client certificates differ from server ones.

WHY THIS SYSTEM NEEDS IT

Everything crosses a network you do not control. Without TLS, the credentials you add in exercise 1 travel in clear text and undo their own purpose.

WHERE YOU USE IT HERE

Exercise 9. The trap is mechanical: TLS must be configured *before* the socket is created, or it silently does nothing.

```
► openssl s_client -connect localhost:8883 -
CAfile ca.crt
```

Git & reading unfamiliar code

ESSENTIAL

WHAT IT IS

Branches, commits, diffs and history — and the habit of reading a codebase you did not write before changing it.

WHY THIS SYSTEM NEEDS IT

You will spend far more time reading than writing. Git history is also evidence: exercise 11 involves searching it for secrets that should never have been committed.

WHERE YOU USE IT HERE

Every exercise. Each one names a reference implementation in the main repository to compare your answer against — after you have attempted it.

```
► git log --oneline -20 · git log --all -p |
grep -i password
```

WHAT NONE OF THESE CAN BE REPLACED BY

The habit of **not believing anything you have not checked**. Every exercise here can be “completed” by changing a config file and assuming it worked. Several of the traps in this book exist precisely to catch that — a command that exits 0 while the broker silently drops the message, a TLS call that does nothing because it ran a line too late, a rate limiter that looks broken because the test was sequential. Curiosity about *why* something passed is the skill this internship is really teaching.

Running the sandbox

No Docker, no containers. You install five ordinary programs, then start four of them in four terminals. Copy the commands exactly — there is nothing to figure out here.

Step 1 — install the pieces (once)

You need **Python 3.10+**, **Node.js 18+**, and three small background services: an MQTT broker, Redis and PostgreSQL.

Ubuntu, Debian or WSL

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip \
    mosquitto mosquitto-clients redis-server postgresql

# Ubuntu starts mosquitto and Redis immediately, on 1883 and 6379 — the
# ports the full cloud-robotics platform needs. Turn them off; you will
# start your own, on different ports, below.
sudo systemctl stop mosquitto redis-server
sudo systemctl disable mosquitto redis-server

# Postgres is worth leaving running, but move it off 5432 first.
sudo sed -i 's/^port = 5432/port = 5433/' /etc/postgresql/*/main/postgresql.conf
sudo systemctl restart postgresql
pg_lsclusters          # the port column should read 5433
```

macOS, with Homebrew

```
brew install python node mosquitto redis postgresql@16

# Postgres on 5433, so it cannot collide with the main platform
echo "port = 5433" >> $(brew --prefix)/var/postgresql@16/postgresql.conf
brew services start postgresql@16

# do NOT "brew services start" mosquitto or redis — you start those yourself
```

Windows: install WSL2 and follow the Ubuntu steps inside it. Running these services natively on Windows is possible but fiddly, and nothing in these exercises needs it.

Step 2 — create the database (once)

```
sudo -u postgres psql -p 5433 -c "CREATE USER robotstore WITH PASSWORD 'robotstore';"
sudo -u postgres psql -p 5433 -c "CREATE DATABASE robotstore OWNER robotstore;"

# on macOS, drop the "sudo -u postgres" — Homebrew's Postgres runs as you:
# psql -p 5433 postgres -c "CREATE USER robotstore WITH PASSWORD 'robotstore';"
```

You never create the tables yourself. The backend does that the first time it starts.

Step 3 — install this project's dependencies (once)

```
cd secure_robotstore

# Python: one virtual environment shared by the backend and the agent
python3 -m venv .venv
source .venv/bin/activate
pip install -r cloud/backend/requirements.txt -r robot-agent/requirements.txt

# JavaScript: the frontend
cd cloud/frontend && npm install && cd ../../
```

A *virtual environment* is a private folder of Python packages belonging to this project alone, so installing something here cannot break anything else on your machine. `source .venv/bin/activate` is what switches a terminal into it — you will do that in two of the four terminals below, every time you open them.

Step 4 — start it, five terminals

Open five terminals, all in the `secure_robotstore` directory. Leave each running; each prints its own log, which is most of how you will debug things.

TERMINAL	COMMAND
1 broker	<code>mosquitto -c mosquitto/mosquitto.local.conf -v</code>
2 redis	<code>redis-server --port 6380</code>
3 backend	<code>source .venv/bin/activate</code> <code>source local.env</code> <code>cd cloud/backend</code> <code>uvicorn app.main:app --reload --port 8001</code>
4 agent	<code>source .venv/bin/activate</code> <code>source local.env</code> <code>cd robot-agent</code> <code>python -m agent.main</code>
5 frontend	<code>cd cloud/frontend</code> <code>VITE_API_URL=http://localhost:8001 npm run dev -- --port 3001</code>

Then open **`http://localhost:3001`** and sign in with `operator` / `demo1234`. Both windows should start filling at one row per second. Postgres is already running in the background as a system service, which is why it gets no terminal of its own.

`source local.env` does something small but essential: the backend and the agent default to the hostnames `mosquitto`, `postgres` and `redis`, which only exist inside a Docker network. That file points them at `localhost` instead.

Where everything is

SERVICE	ADDRESS	NOTES
Frontend	<code>http://localhost:3001</code>	login + Remote Data
Backend API	<code>http://localhost:8001</code>	<code>/docs</code> for the OpenAPI page
Backend WS	<code>ws://localhost:8001/ws/string_api</code>	and <code>/ws/int_api</code>
Agent health	<code>http://localhost:8081</code>	<code>{"mqtt_connected": true}</code>
MQTT broker	<code>localhost:1884</code>	anonymous, for now
PostgreSQL	<code>localhost:5433</code>	<code>robotstore</code> / <code>robotstore</code>
Redis	<code>localhost:6380</code>	no password

Every one of those sits **one above** the full cloud-robotics platform's port (1883 / 6379 / 5432 / 8000 / 3000 / 8080), so the sandbox and the real platform can run on one machine at the same time. Nothing you install for this sandbox should ever listen on the platform's numbers — if the platform suddenly refuses to start, that is almost always what has happened.

If something does not work

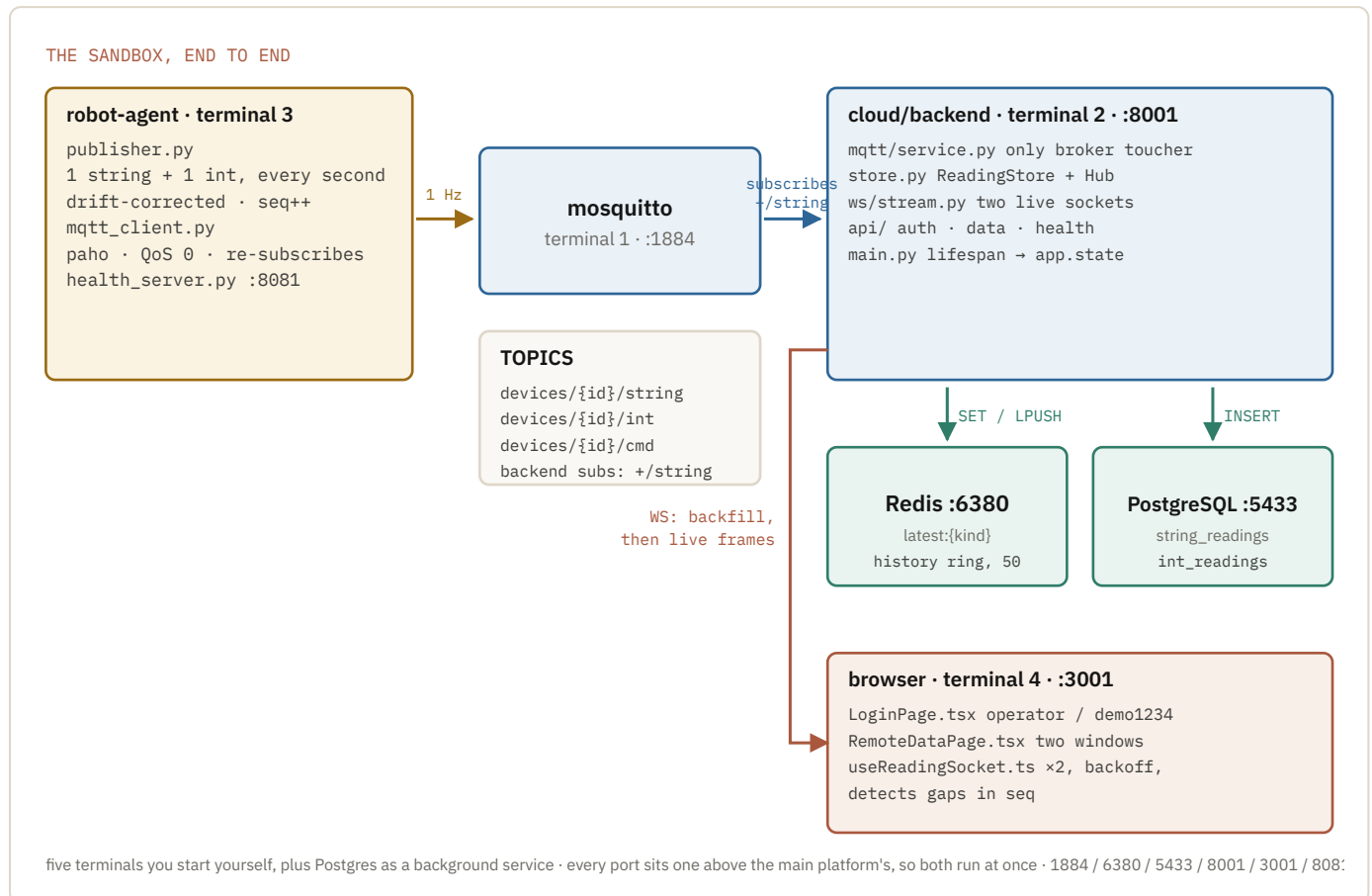
SYMPTOM	WHAT IT MEANS
<code>connection refused</code> on port 1884	Terminal 1 is not running.
The main platform fails with <code>failed to bind host port 127.0.0.1:1883</code>	Ubuntu's auto-started mosquitto or redis service is holding it. <code>sudo systemctl stop mosquitto redis-server</code> , then <code>disable</code> them. This sandbox never uses 1883 or 6379.
Backend log repeats <code>lost the broker connection (rc=7)</code>	Two MQTT clients are using the same client id and kicking each other off. You are running the stack twice — check for an old terminal.
<code>password authentication failed</code>	Step 2 did not run, or ran against a different Postgres. Re-run it.
<code>Peer authentication failed</code>	You connected over the local socket instead of TCP. Add <code>-h localhost</code> to the <code>psql</code> command.
<code>ModuleNotFoundError: fastapi</code>	That terminal is not in the virtual environment. Run <code>source .venv/bin/activate</code> first.
Both windows stay empty	Terminal 2 or 3 is not running. <code>curl localhost:8001/health/detail</code> names the broken dependency.

RESTARTING AFTER A CHANGE

There is no build step and no rebuild. Edit a Python file and the backend reloads itself (`--reload`); edit a React file and the browser updates. Only two things need a manual restart: the broker, after you edit `mosquitto/mosquitto.local.conf` — Ctrl-C in terminal 1 and run it again — and the agent, which has no reloader. That immediacy is the reason this runs without containers: the loop from “edit a config” to “watch the attack fail” is a few seconds.

Architecture overview

One device publishes a string and an integer every second. Everything else in this system exists to move those two numbers safely to a browser — and it is the *safely* that is missing.



One device, two data types, about 1,200 lines. Four processes you start by hand, plus two background services. The architecture is the real one, not a toy: the same MQTT-only device boundary, the same Redis/Postgres split, the same “one service owns the broker connection” rule, and the same paho-thread-to-asyncio hand-off. What is missing is every security control — that is what the twelve exercises put back.

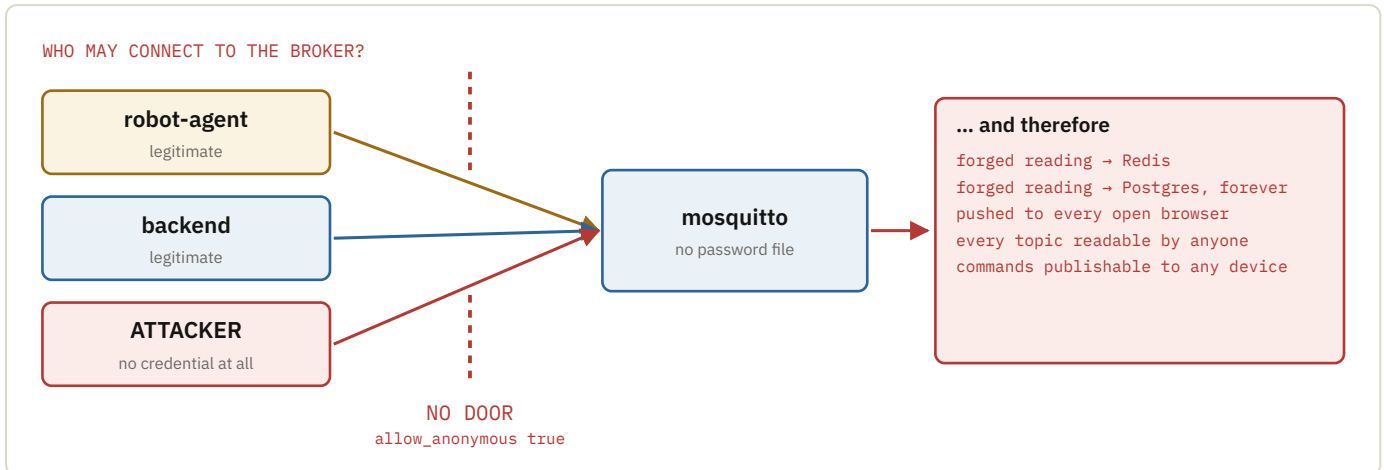
The three design decisions worth understanding first

MQTT is the only road in. Every byte between the device and the cloud crosses the broker — readings northbound, commands southbound. That is the single most valuable security property the architecture has, because it means there is exactly one place to enforce who may say what. Exercises 1, 2, 7 and 9 are all spent on that one place.

Two stores, split by a single question. Postgres is the system of record — losing a row would be a bug. Redis holds what is live and re-derivable: the latest value (another is one second away) and a 50-item ring buffer that populates a browser which has just connected. The test to apply to any new piece of data is *would losing this on restart be a bug?* Yes goes to Postgres; no goes to Redis.

One module owns the broker connection. `mqtt/service.py` is the only backend module that imports paho; everything else reaches the device through it. That is why the WebSocket layer never touches MQTT and the MQTT layer never touches Postgres — and it is why adding TLS and credentials later touches one file rather than five.

One mechanic to know before changing any of it: paho runs its network loop on its own background thread, so its callbacks do *not* fire on the asyncio event loop. Anything a callback needs to do that touches async state is handed back across that boundary with `run_coroutine_threadsafe`. Calling an async handler directly from a paho callback is the classic bug here.



Exercise 1. The broker asks nobody who they are. An attacker who can reach port 1884 is, as far as the system is concerned, exactly as trusted as the robot. Authentication is the foundation everything else rests on: an access-control list cannot scope a client to its own topics if the broker cannot tell clients apart.

WHAT IS WRONG

`mosquitto/mosquitto.conf` sets `allow_anonymous true`. Any client that can open a TCP connection to the broker is a trusted client, and there are no credentials anywhere in this stack to check.

PROVE IT FIRST

```

mosquitto_pub -h localhost -p 1884 -t devices/demo-device-01/int \
  -m '{"value":999,"seq":999999}'

# no credential was offered – and it is now in the system of record
psql -h localhost -p 5433 -U robotstore -d robotstore \
  -c "SELECT * FROM int_readings WHERE seq = 999999;"

```

DONE LOOKS LIKE

- ☐ Anonymous publish and subscribe are both refused by the broker.
- ☐ The agent and the backend hold *separate* credentials, and both still work.
- ☐ No password appears in any committed file or anywhere in git history.
- ☐ A wrong credential produces a clear log line rather than silence.

VERIFY THE FIX

```

# must now fail
mosquitto_pub -h localhost -p 1884 -t devices/demo-device-01/int -m '{"value":1}'

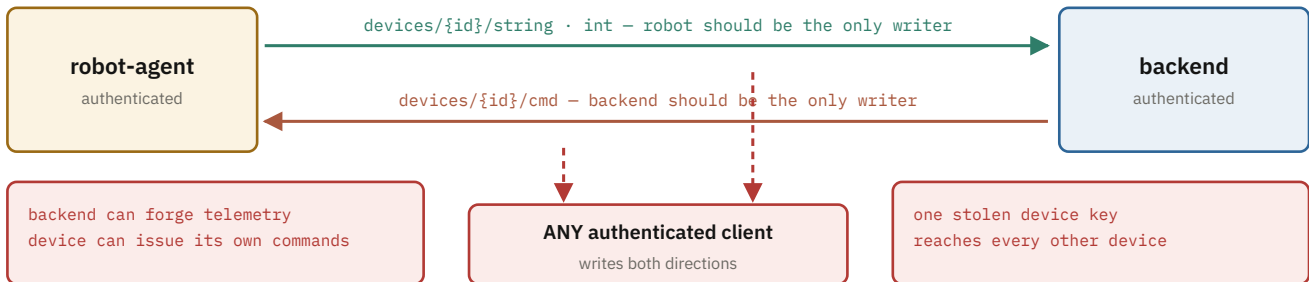
# must still be true – the real clients are unaffected
curl -s localhost:8001/health/detail | python3 -m json.tool
curl -s localhost:8081

```

WATCH FOR paho retries forever in the background, so a wrong password looks like silence rather than an error. Read the broker log.

COMPARE AGAINST `cloud-container/mosquitto/` – note how `docker-entrypoint-wrapper.sh` generates the password file at container start so no plaintext ever enters git.

WHO MAY WRITE WHICH TOPIC?



Exercise 2. Authentication proves *who*; it says nothing about *what they may do*. Without an ACL every authenticated client can write every topic — so a compromised backend can fabricate a robot's own self-reported state, and anything holding the device credential can command the whole fleet. Direction is the security property here.

WHAT IS WRONG

With no `acl_file`, every authenticated client may publish and subscribe to everything. A compromised backend can forge device telemetry, and anything holding the agent's credential can publish commands back to the device.

PROVE IT FIRST

```
# with exercise 1 done, connect as the BACKEND and publish device telemetry.
# it succeeds — and it should not: the backend has no business claiming to be a device.
mosquitto_pub -h localhost -p 1884 -u backend -P "$MQTT_BACKEND_PASSWORD" \
  -t devices/demo-device-01/string -m '{"value":"forged","seq":1}'
```

DONE LOOKS LIKE

- ☐ The backend identity cannot publish telemetry, status or health.
- ☐ The device identity cannot publish to the command topic.
- ☐ Neither identity can subscribe to a topic outside its role.
- ☐ Real telemetry and real commands both still flow.

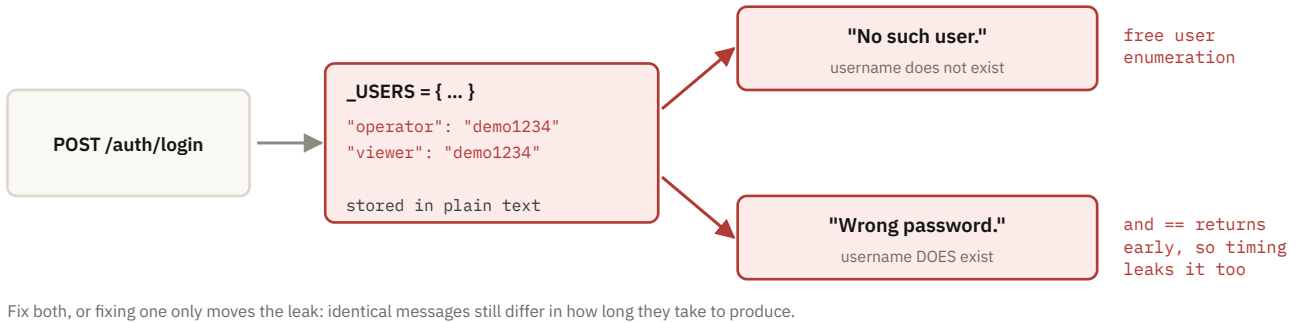
VERIFY THE FIX

```
# both of these must now be refused
mosquitto_pub -h localhost -p 1884 -u backend -P "$MQTT_BACKEND_PASSWORD" \
  -t devices/demo-device-01/string -m '{"value":"forged"}'
mosquitto_pub -h localhost -p 1884 -u demo-device-01 -P "$MQTT_DEVICE_PASSWORD" \
  -t devices/demo-device-01/cmd -m '{"action":"drive"}'
```

STEAL THIS TRICK Use a `pattern` rule with `%u` so each device is scoped to its own namespace automatically. Onboarding device #2 then means issuing a credential, not editing the ACL file.

COMPARE AGAINST `cloud-container/mosquitto/aclfile`

TWO WAYS THE LOGIN HELPS AN ATTACKER



Exercise 3. The endpoint answers a question it was never asked — “does this account exist?” — before any password is tried. Passwords are then compared with `==`, which stops at the first wrong character, so response timing carries the same information even after the messages are made identical.

WHAT IS WRONG

`cloud/backend/app/api/auth.py` holds a dictionary of plaintext passwords in source, compares them with `==`, and returns a different error for “no such user” than for “wrong password”.

PROVE IT FIRST

```
curl -s -X POST localhost:8001/auth/login -H 'Content-Type: application/json' \
  -d '{"username":"operator","password":"wrong"}' # "Wrong password."
curl -s -X POST localhost:8001/auth/login -H 'Content-Type: application/json' \
  -d '{"username":"nobody","password":"wrong"}' # "No such user."
```

DONE LOOKS LIKE

- ☐ Both failing requests return byte-identical responses.
- ☐ Passwords are stored hashed with `bcrypt` or `argon2` — never a bare SHA.
- ☐ The comparison is constant-time, on *both* fields.
- ☐ A failure costs the same work when the user does not exist, so the timing leak does not return through the skipped hash.

VERIFY THE FIX

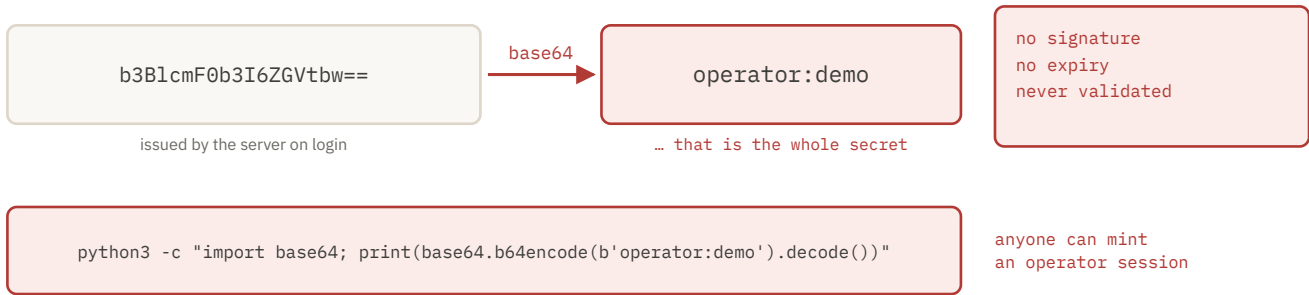
```
# identical bodies...
for u in operator nobody; do
  curl -s -X POST localhost:8001/auth/login -H 'Content-Type: application/json' \
    -d '{"username":"$u","password":"x"}'; echo
done

# ...and indistinguishable timing
for u in operator nobody; do
  curl -s -o /dev/null -w "$u ${time_total}\n" -X POST localhost:8001/auth/login \
    -H 'Content-Type: application/json' -d '{"username":"$u","password":"x"}'
done
```

A CAVEAT ON THE TARGET The reference file checks one configured credential rather than a users table; its own docstring calls a table with hashed passwords the natural extension. Either is a legitimate answer here.

COMPARE AGAINST `cloud-container/backend/app/auth/service.py`

WHAT THE TOKEN ACTUALLY IS



Exercise 4. This is not a token, it is a username in a costume — reversible by anyone, forgeable in one line, and valid forever because nothing in it says when it should stop being. A signature makes it unforgeable; an expiry bounds the damage of a stolen one; a unique id is what later makes revocation possible at all.

WHAT IS WRONG

The token is `base64(username + ":demo")`. It is not signed, so anyone can mint one, and it carries no expiry, so it is valid forever.

PROVE IT FIRST

```
python3 -c "import base64; print(base64.b64encode(b'operator:demo').decode())"

# that is the exact string the server issues – you just minted an
# operator session without touching the server
```

DONE LOOKS LIKE

- ☐ A token with a tampered payload is rejected.
- ☐ A token past its `exp` is rejected, without a restart.
- ☐ Signing and verification are pure functions, unit-testable without starting the app.
- ☐ The signing secret comes from the environment, and the app refuses to start on a known default.

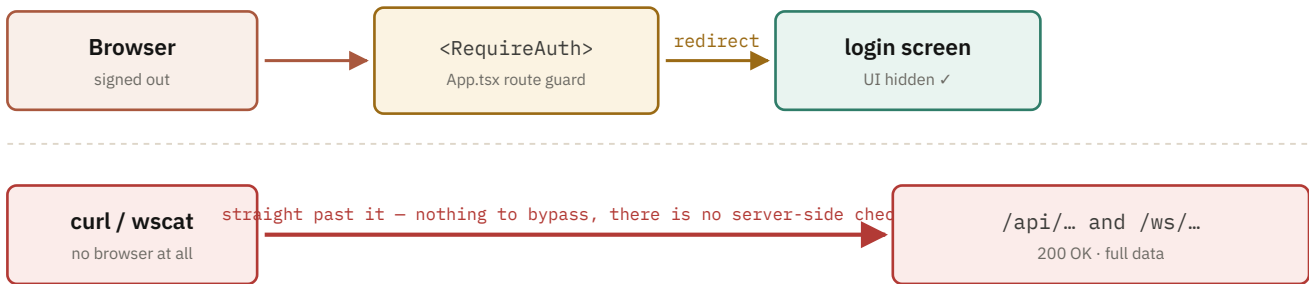
VERIFY THE FIX

```
# pair this with exercise 5 – until a route validates, nothing here is observable
curl -s -o /dev/null -w "%{http_code}\n" localhost:8001/api/int/latest \
-H "Authorization: Bearer $FORGED"      # expect 401
```

GO ONE STEP FURTHER Add a `jti` claim and a Redis revocation list. A JWT is stateless and so cannot be cancelled before it expires; `jti` is the handle that makes logout actually invalidate a session.

COMPARE AGAINST `cloud-container/backend/app/auth/tokens.py`

THE GUARD IS IN THE WRONG PLACE



A route guard is a user-interface convenience. Authorisation is a decision only the server can make.

Exercise 5 — the one that matters most. Exercises 3 and 4 are worth nothing without this: a perfectly hashed password and a perfectly signed token protect nothing if no endpoint ever checks the result. Note the WebSocket half is genuinely harder — a browser cannot set headers on the handshake, so a short-lived single-use ticket travels in the URL instead of the real token.

WHAT IS WRONG

Nothing checks the token. Every REST route in `api/data.py` and both WebSockets in `ws/stream.py` serve anyone. The route guard in `App.tsx` hides the interface and protects nothing. **Exercises 3 and 4 are worth nothing on their own** — a perfect token nobody validates is still a perfect token nobody validates.

PROVE IT FIRST

```

curl -s -o /dev/null -w "%{http_code}\n" localhost:8001/api/string/history

python3 - <<'PY'
import asyncio, websockets
async def main():
    async with websockets.connect("ws://localhost:8001/ws/int_api") as ws:
        print("connected with no token:", (await ws.recv())[:60])
asyncio.run(main())
PY
  
```

DONE LOOKS LIKE

- ☐ Every REST route returns 401 without a valid token.
- ☐ Both WebSocket upgrades are refused before `accept()`, not after.
- ☐ A ticket cannot be used twice, and expires on its own.
- ☐ The browser still works end to end: log in, both windows live.

VERIFY THE FIX

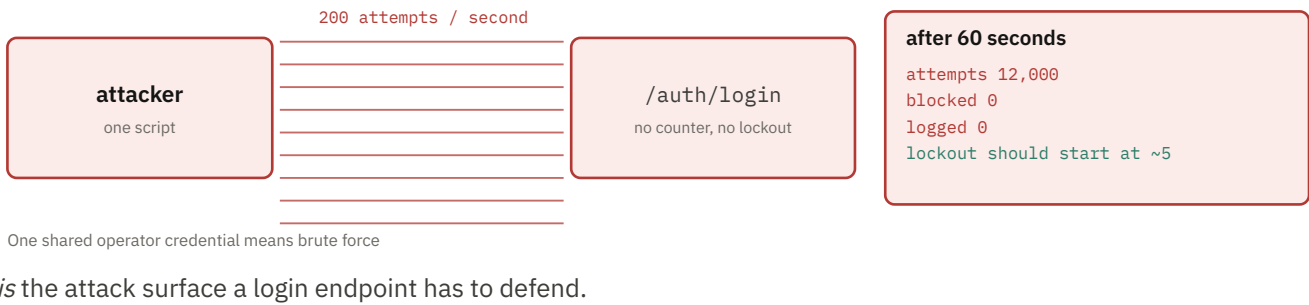
```

# both must fail before a token and succeed after
curl -s -o /dev/null -w "%{http_code}\n" localhost:8001/api/string/history
# and the same ticket must be rejected the second time it is used
  
```

THE REAL CONSTRAINT Browsers cannot set custom headers on a WebSocket handshake, so *something* must travel in the URL. A token there ends up in proxy and access logs — so issue a 15-second single-use ticket instead, consumed atomically with `GETDEL`.

COMPARE AGAINST `cloud-container/backend/app/auth/dependencies.py`

GUESSING COSTS THE ATTACKER NOTHING



Exercise 6. A fixed window is the right amount of machinery here — this is one account being guessed at, not traffic shaping across millions of users. Choosing the simpler correct mechanism is part of the lesson. Watch the side effect: a rate limiter makes a test suite non-idempotent, which is how finding F9 was born.

WHAT IS WRONG

`/auth/login` can be tried as fast as the network allows. There is no counter, no lockout and no record.

PROVE IT FIRST

```
time for i in $(seq 1 200); do
  curl -s -o /dev/null -X POST localhost:8001/auth/login \
    -H 'Content-Type: application/json' \
    -d '{"username":"operator","password":"guess"$i}'
done
```

DONE LOOKS LIKE

- ☐ Repeated failures trigger a lockout that expires on its own.
- ☐ A correct password during the lockout is *also* refused — and you can explain why that is correct.
- ☐ A successful login clears both keys.
- ☐ The existing test suite passes, or the reason it changed is written down.

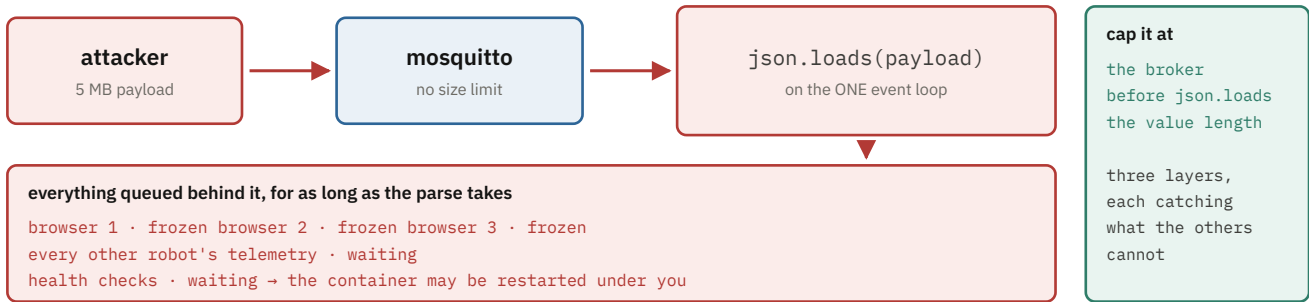
VERIFY THE FIX

```
for i in $(seq 1 30); do
  curl -s -o /dev/null -w "%{http_code} " -X POST localhost:8001/auth/login \
    -H 'Content-Type: application/json' \
    -d '{"username":"operator","password":"guess$i}"
done; echo
# expect 401s, then 429s from the threshold onward
```

CHOOSE THE NUMBERS DELIBERATELY Consider a classroom of thirty students mistyping passwords behind one NAT address. Write down your reasoning for the threshold and the durations.

WATCH FOR A rate limiter makes a test suite non-idempotent — that was finding F9, caused by this exact fix.

ONE BIG MESSAGE STOPS EVERYONE



Exercise 7. The backend parses every inbound message on the single asyncio event loop that serves every WebSocket client in the system. Derive the limit from the largest legitimate message rather than picking a round number — and verify by *delivery*, because `mosquitto_pub` exits 0 even when the broker silently drops the message.

WHAT IS WRONG

Two places accept unbounded input: `mqtt/service.py` calls `json.loads()` on whatever arrives with no size check, and `store.py` writes a string reading of any length to Postgres, Redis and every open browser. `mosquitto.conf` sets no `message_size_limit` either.

PROVE IT FIRST

```
python3 -c "print('{\"value\": \"' + 'A'*5000000 + '\", \"seq\": 424242}')" > /tmp/huge.json
mosquitto_pub -h localhost -p 1884 -t devices/demo-device-01/string -f /tmp/huge.json

# watch the UI while that lands — the parse runs on the single event
# loop serving every WebSocket client, so one message stalls everyone
```

DONE LOOKS LIKE

- ☐ An oversized publish is dropped and never reaches Postgres.
- ☐ Normal readings keep flowing throughout the attack.
- ☐ Limits exist at all three layers: broker, pre-parse byte cap, and value length.
- ☐ A test covers the exact boundary, and another asserts the ceiling itself stays sane.

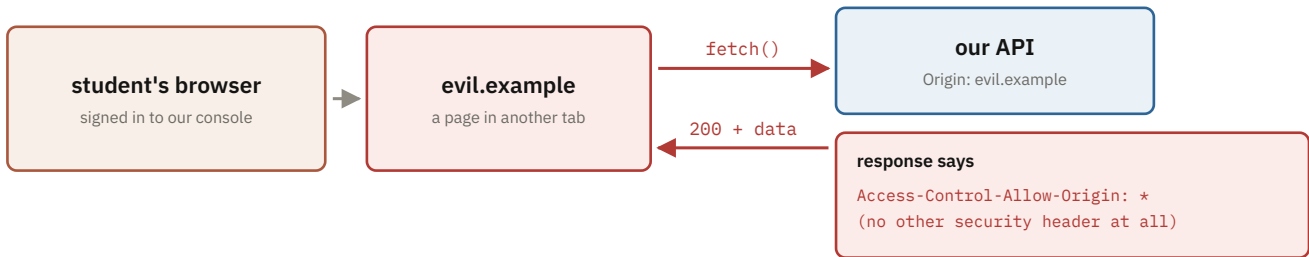
VERIFY THE FIX

```
# verify by DELIVERY — mosquitto_pub exits 0 even when the broker drops it
psql -h localhost -p 5433 -U robotstore -d robotstore \
  -c "SELECT count(*) FROM string_readings WHERE seq = 424242;" # must be 0
curl -s localhost:8001/health/detail # backend still healthy
```

DERIVE, DO NOT GUESS Work the limit out from the largest legitimate message. The main repo's comment reasons from a LiDAR scan (~8 KB) and an SDP offer (~6 KB) to a 256 KB ceiling.

COMPARE AGAINST `cloud-container/backend/app/mqtt/service.py` and the `message_size_limit` block in `cloud-container/mosquitto/mosquitto.conf`

ANY SITE A SIGNED-IN STUDENT VISITS



The headers that are missing — nosniff, frame-deny, CSP, referrer, no-store on the login response — are each cheap, which is why skipping them is hard to justify later.

Exercise 8. A wildcard origin means the browser will hand our API's responses to any page the student happens to have open. Add an explicit allowlist — and for each header you add, write one sentence saying what attack it stops. If you cannot, do not add it: unexplained headers are cargo cult, not security.

WHAT IS WRONG

`CORS_ALLOWED_ORIGINS` is `"*"`, so any site a signed-in user visits can call this API from their browser. Neither the API nor nginx sets a single security header.

PROVE IT FIRST

```

curl -sI localhost:8001/health | grep -iE 'x-frame|x-content|content-security|referrer' \
  || echo "no security headers at all"
curl -s -o /dev/null -w "%{http_code}\n" -H "Origin: https://evil.example" \
  localhost:8001/api/int/latest
  
```

DONE LOOKS LIKE

- ☐ The `evil.example` origin is refused; `http://localhost:3001` still works.
- ☐ Headers are present on both the API and the frontend host.
- ☐ Login responses carry `Cache-Control: no-store`.
- ☐ A one-line justification exists for every header you added.

VERIFY THE FIX

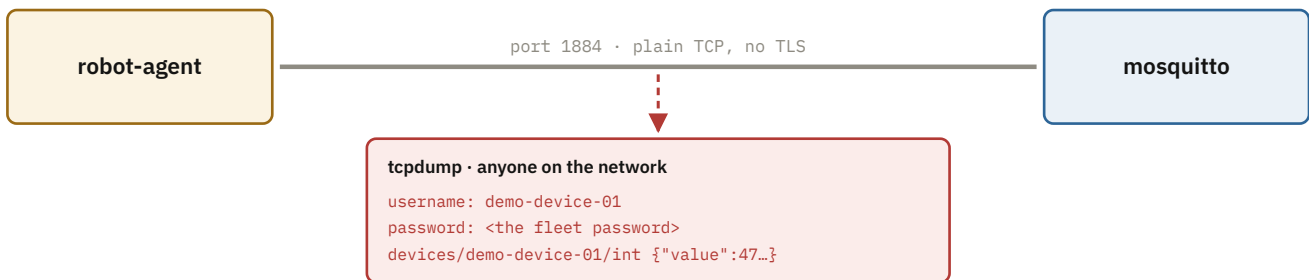
```

curl -sI localhost:8001/health | grep -iE 'x-frame|x-content|content-security|permissions'
curl -sI localhost:3001/ | grep -iE 'x-frame|x-content|content-security'
  
```

KNOW WHAT EACH ONE DOES Never combine a wildcard origin with credentialed requests — find out what browsers do with that combination and why.

COMPARE AGAINST `cloud-container/backend/app/security_headers.py`, `cloud-container/frontend/nginx.conf`

WHAT A LAPTOP ON THE SAME WI-FI SEES



Exercise 1 gave every client a password. Without exercise 9 that password crosses the room in clear text on every reconnect — and exercise 1 is undone.

Exercise 9. Encryption and authentication are not alternatives; either one without the other is a false sense of security. The trap in this task is mechanical rather than conceptual: paho applies TLS when the socket is created, so calling `tls_set()` after `connect_async()` silently does nothing and you get a plaintext connection that looks perfectly healthy.

WHAT IS WRONG

Every hop is plaintext: MQTT on 1884, Postgres, Redis, and HTTP at the edge. Exercise 2 already proved the broker half — and once exercise 1 is done, an eavesdropper reads the *credentials* off the wire on every connect.

PROVE IT FIRST

```

mosquitto_sub -h localhost -p 1884 -t 'devices/#' -v

# or, from anywhere on the network path:
sudo tcpdump -i any -A -c 40 'port 1884' | grep 'devices/'
  
```

DONE LOOKS LIKE

- ☐ Both clients connect over 8883; the plaintext listener is closed or restricted.
- ☐ A packet capture on the TLS port shows no readable topic names or payloads.
- ☐ An untrusted certificate is rejected, with a log line saying so — never silently downgraded.
- ☐ The port and TLS mode are configuration, not code.

VERIFY THE FIX

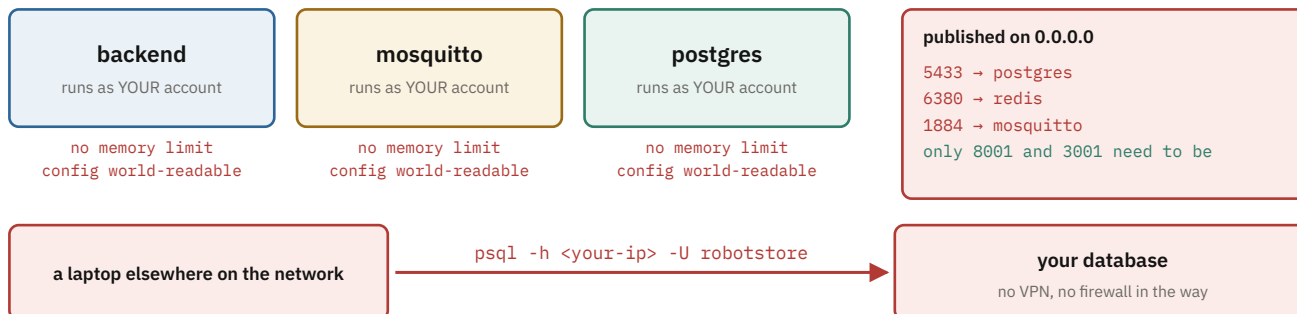
```

sudo tcpdump -i any -A -c 40 'port 1884' | grep -c 'devices/'
sudo tcpdump -i any -A -c 40 'port 8883' | grep -c 'devices/'    # expect 0
  
```

THE MECHANICAL TRAP paho applies TLS when the socket is created, so `tls_set()` must be called *before* `connect_async()`. Call it after and it silently does nothing.

COMPARE AGAINST `scripts/generate-dev-certs.sh`, and the `mqtt_tls_*` settings in `cloud-container/backend/app/config.py`

HOW BIG IS THE BLAST RADIUS?



Exercise 10. These controls do not stop an intrusion; they bound what an intrusion becomes. A service compromised while running as your own account — with your source, your keys and your shell history in reach, and its port open to the whole network — is a very different incident from one running as a dedicated user, memory-capped, listening only on loopback. Same principle the real stack applies with containers; here you apply it directly.

WHAT IS WRONG

Everything you started runs as *you* — your login account, with access to every file you own. Redis and Postgres were installed with defaults, no memory limit is set anywhere, and the broker binds every network interface rather than just loopback.

PROVE IT FIRST

```
whoami                                # the backend runs as this account
ss -tlnp | grep -E '1884|5433|6380'   # bound on 0.0.0.0, not 127.0.0.1

# from another machine on the same network, this reaches your database:
psql -h <this-host> -p 5433 -U robotstore -d robotstore
```

DONE LOOKS LIKE

- ☐ The broker, Redis and Postgres accept connections only from `127.0.0.1`.
- ☐ No service runs as root, and none runs as an account that owns your source code.
- ☐ Config files holding credentials are readable only by their owner.
- ☐ A memory limit exists for each service, derived from a measurement rather than a guess.

VERIFY THE FIX

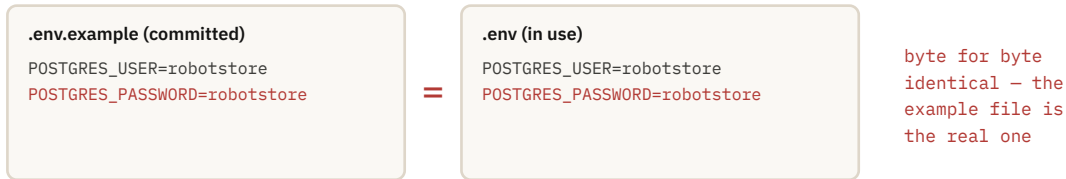
```
ss -tlnp | grep -E '1884|5433|6380'   # expect 127.0.0.1, never 0.0.0.0
ps -o user=,cmd= -C mosquitto -C redis-server # expect a dedicated user
ls -l mosquitto/passwordfile           # expect -rw-----
```

EXPECT BREAKAGE Narrowing a bind address or changing a service user usually breaks something that was quietly relying on the looser setting. Fix it properly rather than by widening the permission again.

THE SAME IDEA, CONTAINERISED The real stack solves this with non-root `USER` directives, `no-new-privileges` and loopback-only port publishing — see the root `docker-compose.yml` and `cloud-container/docker/backend.Dockerfile`. The principles are identical; only the mechanism differs.

11 Secrets

EVERY DEPLOYMENT SHARES ONE PASSWORD



school A

same credentials

school B

same credentials

school C

same credentials

generate per environment
refuse to boot on a default
when ENVIRONMENT=production

Exercise 11. The half people skip is the startup guard, and it is the half that works. A default credential which only fails at code-review time will eventually reach production, because review is a human process that misses things; one that refuses to boot cannot. Check git history too — anything already committed is exposed and must be rotated, not merely deleted.

WHAT IS WRONG

`.env` and `.env.example` are identical, and both contain `POSTGRES_PASSWORD=robotstore`. `local.env` reuses those defaults verbatim, so every copy of this project shares credentials with every other.

PROVE IT FIRST

```
diff .env .env.example && echo "identical – the example file IS the real one"

# and check whether anything sensitive was ever committed
git log --all -p | grep -iE 'password|secret|api[_-]?key' | grep -v 'example'
```

DONE LOOKS LIKE

- ☐ A fresh checkout cannot start without generating secrets first.
- ☐ A production start carrying any known default fails loudly, naming the setting.
- ☐ Local development still starts with no manual steps beyond the generator.
- ☐ Git history has been checked, and anything found is *rotated*, not just deleted.

VERIFY THE FIX

```
ENVIRONMENT=production POSTGRES_PASSWORD=robotstore \
  uvicorn app.main:app --port 8001
# expect: immediate exit, naming POSTGRES_PASSWORD as an insecure default
```

THE HALF PEOPLE SKIP A default that only fails at code review will eventually reach production. One that refuses to boot cannot.

COMPARE AGAINST `scripts/generate-secrets.sh`, and `assert_production_safe()` with its `_INSECURE_DEFAULTS` set in `cloud-container/backend/app/config.py`

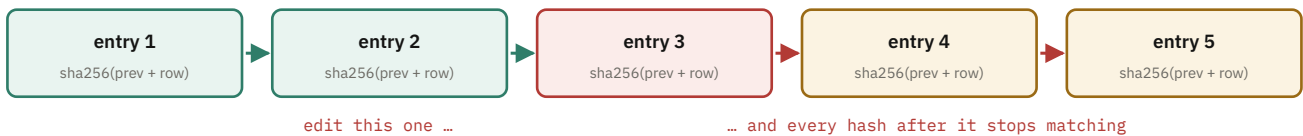
12 Audit logging

CAN YOU PROVE THE RECORD WAS NOT EDITED?

TODAY – a line in the application log



HASH-CHAINED – every entry commits to the one before it



Exercise 12. After an incident involving a machine that moved, the first question is who commanded it. Chaining each entry to the hash of the previous one makes tampering detectable by recomputation alone. Two details carry the weight: canonical serialisation with sorted keys, so identical data always hashes identically, and a database advisory lock around appends, so two concurrent writers cannot both chain from the same predecessor.

WHAT IS WRONG

Auth events reach `logger.info()` and nothing else. That is neither durable — log rotation, a restarted container — nor tamper-evident: anyone with disk access edits it leaving no trace. For a system that drives physical hardware, “who commanded what, when” has to survive an incident review.

PROVE IT FIRST

```
# look at terminal 2, where the backend is running
grep 'login ok' <whatever you scrolled past in that terminal>

# now ask: where does this live in an hour? who could have edited it?
# what would prove that nobody did?
```

DONE LOOKS LIKE

- ☐ Log in, log out and fail a login — all three appear, with actor, action, result and time.
- ☐ The verifier passes on an untouched chain.
- ☐ A single edited, deleted or reordered row is detected *and located*.
- ☐ Concurrent writes do not corrupt the chain under load.

VERIFY THE FIX

```
python3 scripts/verify-audit-log.py          # chain intact, N entries

psql -h localhost -p 5433 -U robotstore -d robotstore \
  -c "UPDATE audit_log SET actor='someone-else' WHERE id = 5;"

python3 scripts/verify-audit-log.py          # break detected at entry 5
```

TWO DETAILS CARRY THE WEIGHT Canonical JSON with sorted keys, so identical data always hashes identically; and a Postgres advisory lock around appends, so two concurrent writers cannot both chain from the same predecessor.

COMPARE AGAINST `cloud-container/backend/app/audit/logger.py`

When you are done

Re-run the three headline checks one final time.

```
make verify-holes
```

All three should now fail to demonstrate anything — the anonymous publish refused, the subscribe refused, the unauthenticated API read returning 401 — while the browser at `http://localhost:3001` still shows both windows ticking once a second.

THAT COMBINATION IS THE ACTUAL PASS CONDITION

Either half alone is easy. Anyone can lock a system down until it stops working, and anyone can keep a system working by changing nothing. Doing both at once, and being able to prove both at once, is the entire job.

What is deliberately still missing

These are real limitations of the sandbox, and worth understanding before anyone builds on it. The `Hub` is in-process, so two backend replicas would each only push to their own WebSocket clients — the fix is Redis pub/sub between them. There are no tests; adding one for the reading path is a good first non-security task. There are no migrations, only `CREATE TABLE IF NOT EXISTS`. `--reload` and bind-mounted source are on for both Python services so your edits apply immediately — development settings, not production ones. And Postgres grows forever at about two rows per second; retention is not handled.

None of those are security problems. Being able to tell the difference — and to say so plainly rather than filing everything as a vulnerability — is part of what makes a security engineer worth listening to.

The Twelve Exercises — `secure_robotstore`, a training mirror of the cloud-robotics teleoperation platform. Companion material: `secure_robotstore/README.md`, `docs/security-findings.md` (the audit these exercises draw their real findings from), and `docs/target-architecture.md`.